SQL fundamentals

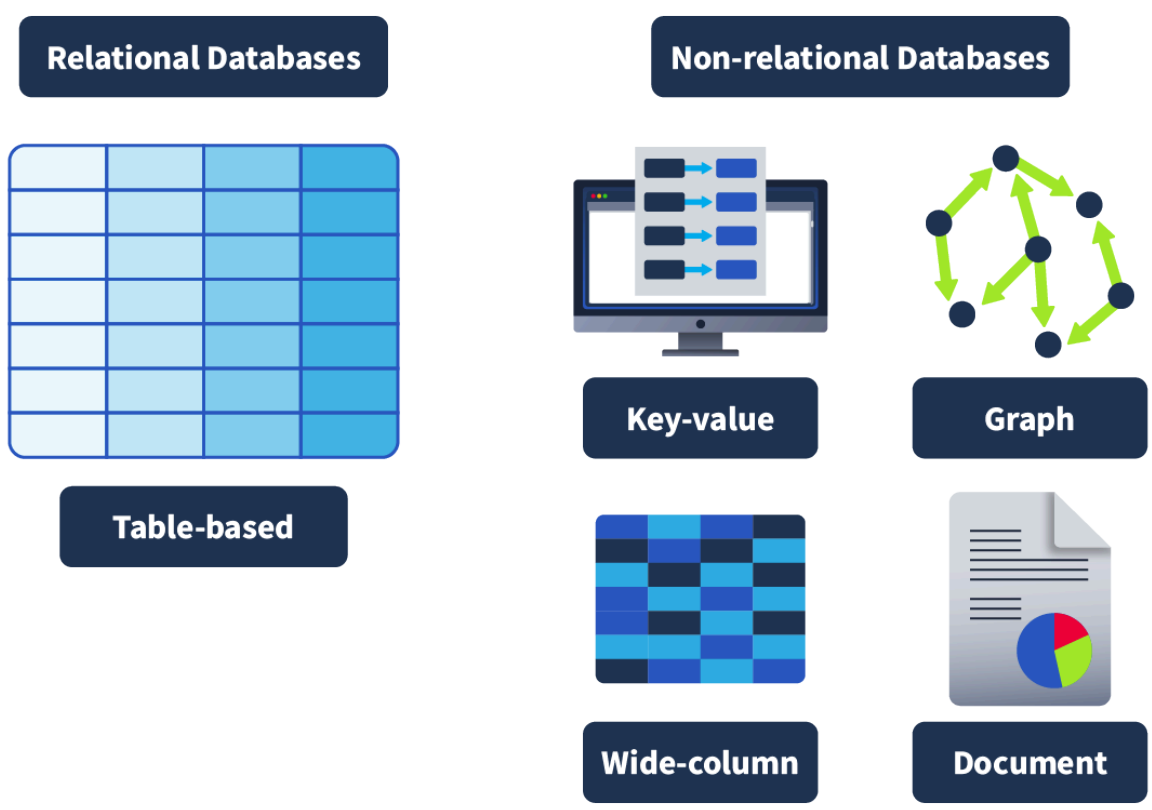
Introducing Databases

Okay, so you've been told just how important they are. Now, it's time to understand what they are in the first place. As mentioned in the introduction, databases are so ubiquitous that you very likely interact with systems that are using them. Databases are an organised collection of structured information or data that is easily accessible and can be manipulated or analysed. That data can take many forms, such as user authentication data (such as usernames and passwords), which are stored and checked against when authenticating into an application or site (like TryHackMe, for example), user-generated data on social media (Like Instagram and Facebook) where data such as user posts, comments, likes etc are collected and stored, as well as information such as watch history which is stored by streaming services such as Netflix and used to generate recommendations.

I'm sure you get the point: databases are used extensively and can contain many different things. It's not just massive-scale businesses that use databases. Smaller-scale businesses, when setting up, will almost certainly have to configure a database to store their data. Speaking of kinds of databases, let's take a look now at what those are.

Different Types of Databases

Now it makes sense that something is used by so many and for (relatively) so long that there would be multiple types of implementations. There are quite a few different types of databases that can be built, but for this introductory room, we are going to focus on the two primary types: **relational databases** (aka SQL) vs **non-relational databases** (aka NoSQL).



Relational databases: Store structured data, meaning the data inserted into this database follows a structure. For example, the data collected on a user consists of first_name, last_name, email_address, username and password. When a new user joins, an entry is made in the database following this structure. This structured data is stored in rows and columns in a table (all of which will be covered shortly); relationships can then be made between two or more tables (for example, user and order_history), hence the term relational databases.

Non-relational databases: Instead of storing data the above way, store data in a non-tabular format. For example, if documents are being scanned, which can contain varying types and quantities of data, and are stored in a database that calls for a non-tabular format. Here is an example of what that might look like:

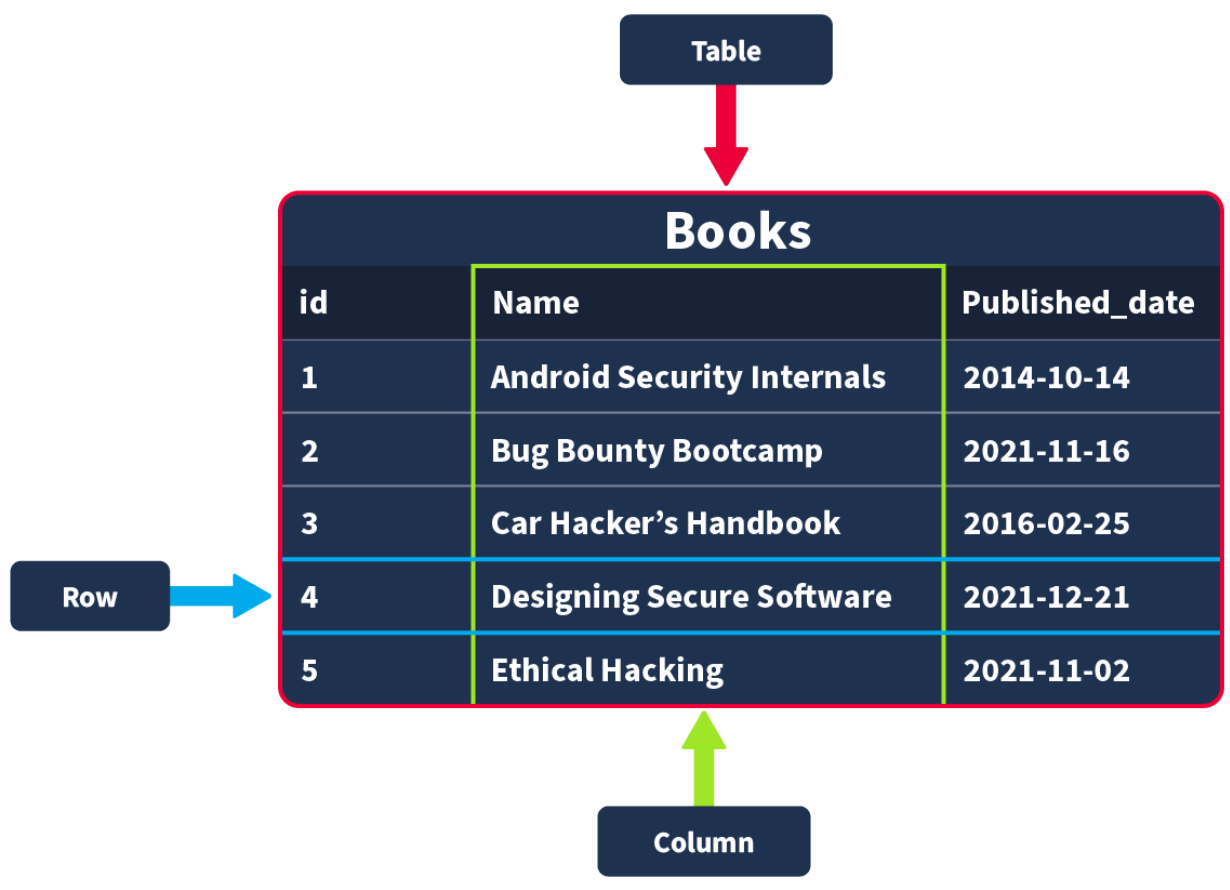
```
{
   _id: ObjectId("4556712cd2b2397ce1b47661"),
   name: { first: "Thomas", last: "Anderson" },
   date_of_birth: new Date('Sep 2, 1964'),
   occupation: [ "The One"],
```

```
    steps_taken : NumberLong(4738947387743977493)
}
```

In terms of what database should be chosen, it always comes down to the context in which the database is going to be used. Relational databases are often used when the data being stored is reliably going to be received in a consistent format, where accuracy is important, such as when processing e-commerce transactions. Non-relational databases, on the other hand, are better used when the data being received can vary greatly in its format but need to be collected and organised in the same place, such as social media platforms collecting user-generated content.

Tables, Rows and Columns

Now that we've defined the two primary types of databases, we'll focus on relational databases. We'll start by explaining **tables**, **rows**, and **columns**. All data stored in a relational database will be stored in a **table**; for example, a collection of books in stock at a bookstore might be stored in a table named "Books".

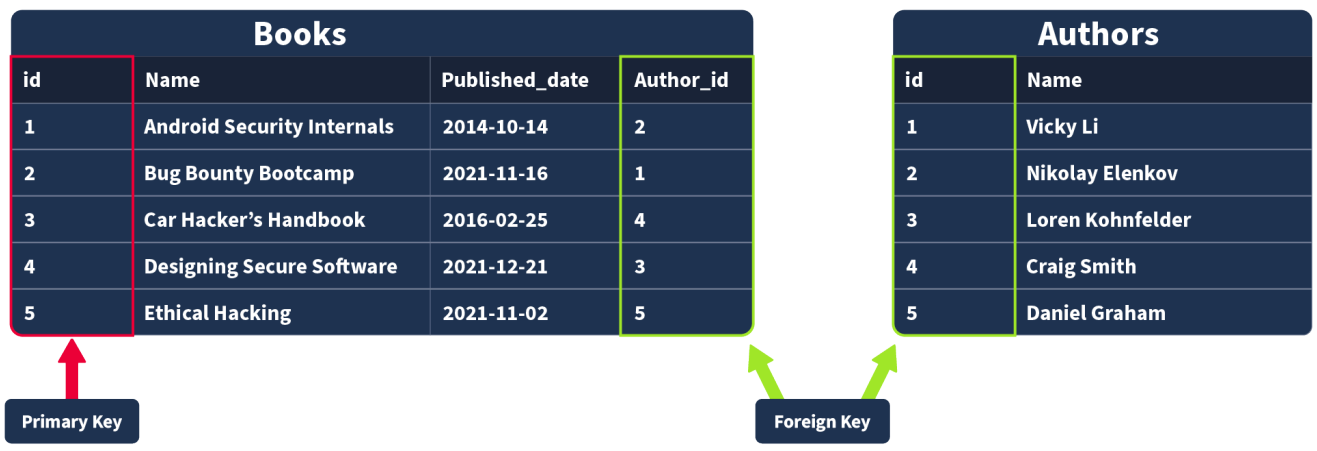


When creating this table, you would need to define what pieces of information are needed to define a book record, for example, "id", "Name", and "Published_date". These would then be your **columns**; when these columns are being defined, you would also define what data type this column should contain; if an attempt is made to insert a record into a database where the data type does not match, it is rejected. The data types that can be defined can vary depending on what database you are using, but the core data types used by all include Strings (a collection of words and characters), Integers (numbers), floats/decimals (numbers with a decimal point) and Times/Dates.

Once a table has been created with the columns defined, the first record would be inserted into the database, for example, a book named "Android Security Internals" with an id of "1" and a publication date of "2014-10-14". Once inserted, this record would be represented as a **row**.

Primary and Foreign Keys

Once a table has been defined and populated, more data may need to be stored. For instance, we want to create a table named "Authors" that stores the authors of the books sold in the store. Here is a very clear example of a relationship. A book (stored in the Books table) is written by an author (stored in the Authors table). If we wanted to query for a book in our story but also have the author of that book returned, our data would need to be related somehow; we do this with keys. There are two types of **keys**:



Primary Keys: A primary key is used to ensure that the data collected in a certain column is unique. That is, there needs to be a way to identify each record stored in a table, a value unique to that record and is not repeated by any other record in that table. Think about matriculation numbers in a university; these are numbers assigned to a student so they can be uniquely identified in records (as sometimes students can have the same name). A column has to be chosen in each table as a primary key; in our example, "id" would make the most sense as an id has been uniquely created for each book where, as books can have the same publication date or (in rarer cases) book title. Note that there can only be one primary key column in a table.

Foreign Keys: A foreign key is a column (or columns) in a table that also exists in another table within the database, and therefore provides a link between the two tables. In our example, think about adding an "author_id" field to our "Books" table; this would then act as a foreign key because the author_id in our Books table corresponds to the "id" column in the author table. Foreign keys are what allow the relationships between different tables in relational databases. Note that there can be more than one foreign key column in a table.

What is SQL?

Now, all of this theoretically sounds great, but in practice, how do databases work? How would you go and make your first table and populate it with data? What would you use? Databases are usually controlled using a Database Management System (DBMS). Serving as an interface between the end user and the database, a DBMS is a software program that allows users to retrieve, update and manage the data being stored. Some examples of DBMSs include MySQL, MongoDB, Oracle Database and Maria DB.

The interaction between the end user and the database can be done using SQL (Structured Query Language). SQL is a programming language that can be used to query, define and manipulate the data stored in a relational database.

The Benefits of SQL and Relational Databases

SQL is almost as ubiquitous as databases themselves, and for good reason. Here are some of the benefits that come with learning and using to use SQL:

- **It's *fast*:** Relational databases (aka those that SQL is used for) can return massive batches of data almost instantaneously due to how little storage space is used and high processing speeds.
- **Easy to Learn:** Unlike many programming languages, SQL is written in plain English, making it much easier to pick up. The highly readable nature of the language means users can concentrate on learning the functions and syntax.
- **Reliable:** As mentioned before, relational databases can guarantee a level of accuracy when it comes to data by defining a strict structure into which data sets must fall in order to be inserted.
- **Flexible:** SQL provides all kinds of capabilities when it comes to querying a database; this allows users to perform vast data analysis tasks very efficiently.

Database Statements

CREATE DATABASE

If a new database is needed, the first step you would take is to create it. This can be done in SQL using the `CREATE DATABASE` statement. This would be done using the following syntax:

Terminal

```
mysql> CREATE DATABASE database_name;
```

Run the following command to create a database named `thm_bookmarket_db`:

Terminal

```
mysql> CREATE DATABASE thm_bookmarket_db;
```

SHOW DATABASES

Now that we have created a database, we can view it using the `SHOW DATABASES` statement. The `SHOW DATABASES` statement will return a list of present databases. Run the statement as follows:

Terminal

```
mysql> SHOW DATABASES;
```

In the returned list, you should see the database you have just created and some databases that are included by default (mysql, information_scheme, performance_scheme and sys), which are used for various purposes that enable mysql to function. Also present are various tables needed for this lesson.

USE DATABASE

Once a database is created, you may want to interact with it. Before we can interact with it, we need to tell mysql which database we would like to interact with (so it knows which database to run subsequent queries against). To set the database we have just created as the active database, we would run the `USE` statement as follows (make sure to run this on your machine):

Terminal

```
mysql> USE thm_bookmarket_db;
```

DROP DATABASE

Once a database is no longer needed (maybe it was created for test purposes, or is no longer required), it can be removed using the `DROP` statement. To remove a database, we would use the following statement syntax (although, in our case, we want to keep our database, so no need to run this one yourself!):

Terminal

```
mysql> DROP database database_name;
```

Table Statements

Now that you can create, list, use, and remove databases, it's time to examine how we would populate those databases with tables and interact with those tables.

CREATE TABLE

Following the logic of the database statements, creating tables also uses a `CREATE` statement. Once a database is active (you have run the `USE` statement on it), a table can be created within it using the following statement syntax:

Terminal

```
mysql> CREATE TABLE example_table_name (
    example_column1 data_type,
    example_column2 data_type,
    example_column3 data_type
);
```

As you can see, there is a little more involved here. In the Databases 101 task, we covered how and when a table is created; it must be decided what columns will make up a record in that table, as well as what data type is expected to be contained within that column. That is what is represented by this syntax here. In the example, there are 3 example columns, but SQL supports many (over 1000). Let's try populating our `thm_bookmarket_db` with a table using the following statement:

Terminal

```
mysql> CREATE TABLE book_inventory (
    book_id INT AUTO_INCREMENT PRIMARY KEY,
    book_name VARCHAR(255) NOT NULL,
    publication_date DATE
);
```

This statement will create a table `book_inventory` with three columns: `book_id`, `book_name` and `publication_date`. `book_id` is an `INT` (Integer) as it should only ever be a number, `AUTO_INCREMENT` is present, meaning the first book inserted would be assigned book_id 1, the second book inserted would be assigned a book_id of 2, and so on. Finally, `book_id` is set as the `PRIMARY KEY` as it will be the way we uniquely identify a book record in our table (and a primary must be present in a table).

Book_name has the data type `VARCHAR(255)`, meaning it can use variable characters (text/numbers/punctuation) and a limit of 255 characters is set and `NOT NULL`, meaning it cannot be empty (so if someone tried to insert a record into this table but the book_name was empty it would be rejected. Publication_date is set as the data type `DATE`.

SHOW TABLES

Just as we can list databases using a SHOW statement, we can also list the tables in our currently active database (the database on which we last used the USE statement). Run the following command, and you should see the table you have just created:

Terminal

```
mysql> SHOW TABLES;
```

DESCRIBE

If we want to know what columns are contained within a table (and their data type), we can describe them using the `DESCRIBE` command (which can also be abbreviated to `DESC`). Describe the table you have just created using the following command:

Terminal

```
mysql> DESCRIBE book_inventory;
```

This will give you a detailed view of the table like so:

DROP Syntax

```
mysql> DESCRIBE book_inventory;
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| book_id          | int          | NO   | PRI | NULL    | auto_increment |
| book_name        | varchar(255) | NO   |     | NULL    |                |
| publication_date | date         | YES  |     | NULL    |                |
+------------------+--------------+------+-----+---------+----------------+
3 rows in set (0.02 sec)
```

ALTER

Once you have created a table, there may come a time when your need for the dataset changes, and you need to alter the table. This can be done using the `ALTER` statement. Let's now imagine that we have decided that we actually want to have a column in our book inventory that has the page count for each book. Add this to our table using the following statement:

Terminal

```
mysql> ALTER TABLE book_inventory
ADD page_count INT;
```

The `ALTER` statement can be used to make changes to a table, such as renaming columns, changing the data type in a column or removing a column.

DROP

Similar to removing a database, you can also remove tables using the `DROP` statement. We don't need to do this, but the syntax you would use for this is:

Terminal

```
mysql> DROP TABLE table_name;
```

CRUD

CRUD stands for **C**reate, **R**ead, **U**pdate, and **D**elete, which are considered the basic operations in any system that manages data.

Let's explore all these different operations when working with **MySQL**. In the next two tasks, we will be using the **books** table that is part of the database **thm_books**. We can access it with the statement `use thm_books;`.

Create Operation (INSERT)

The **Create** operation will create new records in a table. In MySQL, this can be achieved by using the statement `INSERT INTO`, as shown below.

Terminal

```
mysql> INSERT INTO books (id, name, published_date, description)
    VALUES (1, "Android Security Internals", "2014-10-14", "An In-Depth Guide to Andro
id's Security Architecture");

Query OK, 1 row affected (0.01 sec)
```

As we can observe, the `INSERT INTO` statement specifies a table, in this case, **books**, where you can add a new record; the columns **id**, **name**, **published_date**, and **description** are the records in the table. In this example, a new record with an **id** of **1**, a **name** of **"Android Security Internals**", a **published_date** of "**2014-10-14**", and a **description** stating "**Android Security Internals provides a complete understanding of the security internals of Android devices**" was added.

Note: This operation already exists in the database so there is no need to run the query.

Read Operation (SELECT)

The **Read** operation, as the name suggests, is used to read or retrieve information from a table. We can fetch a column or all columns from a table with the `SELECT` statement, as shown in the next example.

Terminal

```
mysql> SELECT * FROM books;
+----+----------------------------+----------------+---------------------------------
--------------------+
| id | name                       | published_date | description
|
+----+----------------------------+----------------+---------------------------------
--------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture |
+----+----------------------------+----------------+---------------------------------
--------------------+

1 row in set (0.00 sec)
```

The above output `SELECT` statement is followed by an `*` symbol indicating that all columns should be retrieved, followed by the `FROM` clause and the table name, in this case, **books**.

If we want to select a specific column like the **name** and **description**, we should specify them instead of the `*` symbol, as shown below.

Terminal

```
mysql> SELECT name, description FROM books;
+----------------------------+-------------------------------------------------------+
| name                       | description                                           |
+----------------------------+-------------------------------------------------------+
| Android Security Internals | An In-Depth Guide to Android's Security Architecture |
+----------------------------+-------------------------------------------------------+

1 row in set (0.00 sec)
```

Update Operation (UPDATE)

The **Update** operation modifies an existing record within a table, and the same statement, `UPDATE`, can be used for this.

Terminal

```
mysql> UPDATE books
    SET description = "An In-Depth Guide to Android's Security Architecture."
    WHERE id = 1;

Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

The `UPDATE` statement specifies the table, in this case, **books**, and then we can use `SET` followed by the column name we will update. The `WHERE` clause specifies which row to update when the clause is met, in this case, the one with **id 1**.

Delete Operation (DELETE)

The **delete** operation removes records from a table. We can achieve this with the `DELETE` statement.

Note: There is no need to run the query. Deleting this entry will affect the rest of the examples in the upcoming tasks.

Terminal

```
mysql> DELETE FROM books WHERE id = 1;

Query OK, 1 row affected (0.00 sec)
```

Above, we can observe the `DELETE` statement followed by the `FROM` clause, which allows us to specify the table where the record will be removed, in this case, **books**, followed by the `WHERE` clause that indicates that it should be the one where the **id** is **1**.

Summary

In summary, **CRUD** operations results are fundamental for data operations and when interacting with databases. The statements associated with them are listed below.

- **Create (INSERT statement)** - Adds a new record to the table.
- **Read (SELECT statement)** - Retrieves record from the table.
- **Update (UPDATE statement)** - Modifies existing data in the table.
- **Delete (DELETE statement)** - Removes record from the table.

These operations enable us to effectively manage and manipulate data within a database.

A clause is a part of a statement that specifies the criteria of the data being manipulated, usually by an initial statement. Clauses can help us define the type of data and how it should be retrieved or sorted.

In previous tasks, we already used some clauses, such as `FROM` that is used to specify the table we are accessing with our statement and `WHERE`, which specifies which records should be used.

This task will focus on other clauses: `DISTINCT`, `GROUP BY`, `ORDER BY`, and `HAVING`.

In this task, we will continue to use the **books** table that is part of the database **thm_books**. We can access it with the statement `use thm_books;`.

DISTINCT Clause

The `DISTINCT` clause is used to avoid duplicate records when doing a query, returning only unique values.

Let's use a query `SELECT * FROM books` and observe the results below.

Terminal

```
mysql> SELECT * FROM books;
+----+----------------------------+----------------+----------------------------------
----------------------+
| id | name                       | published_date | description
```

```
|
+----+----------------------------+----------------+----------------------------------
----------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture   |
|  2 | Bug Bounty Bootcamp        | 2021-11-16     | The Guide to Finding and Reportin
g Web Vulnerabilities |
|  3 | Car Hacker's Handbook      | 2016-02-25     | A Guide for the Penetration Teste
r                     |
|  4 | Designing Secure Software  | 2021-12-21     | A Guide for Developers
|
|  5 | Ethical Hacking            | 2021-11-02     | A Hands-on Introduction to Breaki
ng In                 |
|  6 | Ethical Hacking            | 2021-11-02     |
|
+----+----------------------------+----------------+----------------------------------
----------------------+

6 rows in set (0.00 sec)
```

The query's output displays all the content of the table **books**, and the record **Ethical Hacking** is displayed twice. Let's perform the query again, but this time, using the `DISTINCT` clause.

Terminal

```
mysql> SELECT DISTINCT name FROM books;
+----------------------------+
| name                       |
+----------------------------+
| Android Security Internals |
| Bug Bounty Bootcamp        |
| Car Hacker's Handbook      |
| Designing Secure Software  |
| Ethical Hacking            |
+----------------------------+

5 rows in set (0.00 sec)
```

The output shows that only five rows are returned, and just one instance of the **Ethical Hacking** record is displayed.

GROUP BY Clause

The `GROUP BY` clause aggregates data from multiple records and **groups** the query results in columns. This can be helpful for aggregating functions.

Terminal

```
mysql> SELECT name, COUNT(*)
    FROM books
    GROUP BY name;
+----------------------------+----------+
| name                       | COUNT(*) |
+----------------------------+----------+
| Android Security Internals |        1 |
| Bug Bounty Bootcamp        |        1 |
| Car Hacker's Handbook      |        1 |
| Designing Secure Software  |        1 |
| Ethical Hacking            |        2 |
+----------------------------+----------+
```

```
5 rows in set (0.00 sec)
```

In the example above, the records on the **book** table are regrouped by the result of the `COUNT` function. We already know that **Ethical hacking** is listed twice, so the total **count** is 2, placed at the end since it is **grouped by** count.

ORDER BY Clause

The `ORDER BY` clause can be used to sort the records returned by a query in ascending or descending order. Using functions like `ASC` and `DESC` can help us to accomplish that, as shown below in the next two examples.

ASCENDING ORDER

Terminal

```
mysql> SELECT *
    FROM books
    ORDER BY published_date ASC;
+----+----------------------------+----------------+----------------------------------
----------------------+
| id | name                       | published_date | description
|
+----+----------------------------+----------------+----------------------------------
----------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture   |
|  3 | Car Hacker's Handbook      | 2016-02-25     | A Guide for the Penetration Teste
r                     |
|  5 | Ethical Hacking            | 2021-11-02     | A Hands-on Introduction to Breaki
ng In                 |
|  6 | Ethical Hacking            | 2021-11-02     |
|
|  2 | Bug Bounty Bootcamp        | 2021-11-16     | The Guide to Finding and Reportin
g Web Vulnerabilities |
|  4 | Designing Secure Software  | 2021-12-21     | A Guide for Developers
|
+----+----------------------------+----------------+----------------------------------
----------------------+

6 rows in set (0.00 sec)
```

DESCENDING ORDER

Terminal

```
mysql> SELECT *
    FROM books
    ORDER BY published_date DESC;
+----+----------------------------+----------------+----------------------------------
----------------------+
| id | name                       | published_date | description
|
+----+----------------------------+----------------+----------------------------------
----------------------+
|  4 | Designing Secure Software  | 2021-12-21     | A Guide for Developers
|
|  2 | Bug Bounty Bootcamp        | 2021-11-16     | The Guide to Finding and Reportin
g Web Vulnerabilities |
|  5 | Ethical Hacking            | 2021-11-02     | A Hands-on Introduction to Breaki
ng In                 |
|  6 | Ethical Hacking            | 2021-11-02     |
```

```
|
|  3 | Car Hacker's Handbook      | 2016-02-25     | A Guide for the Penetration Teste
r                     |
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture   |
+----+----------------------------+----------------+----------------------------------
----------------------+

6 rows in set (0.00 sec)
```

We can observe the difference when sorting by ascending order using `ASC` and in descending order using `DESC`, both using the **published_date** as reference.

HAVING Clause

The `HAVING` clause is used with other clauses to filter groups or results of records based on a condition. In the case of `GROUP BY`, it evaluates the condition to `TRUE` or `FALSE`, unlike the `WHERE` clause `HAVING` filters the results after the aggregation is performed.

Terminal

```
mysql> SELECT name, COUNT(*)
    FROM books
    GROUP BY name
    HAVING name LIKE '%Hack%';
+-----------------------+----------+
| name                  | COUNT(*) |
+-----------------------+----------+
| Car Hacker's Handbook |        1 |
| Ethical Hacking       |        2 |
+-----------------------+----------+

2 rows in set (0.00 sec)
```

When working with **SQL** and dealing with logic and comparisons, **operators** are our way to filter and manipulate data effectively. Understanding these operators will help us to create more precise and powerful queries. In the next two tasks, we will be using the **books** table that is part of the database **thm_books2**. We can access it with the statement `use thm_books2;`.

Logical Operators

These operators test the truth of a condition and return a boolean value of `TRUE` or `FALSE`. Let's explore some of these operators next.

LIKE Operator

The `LIKE` operator is commonly used in conjunction with clauses like `WHERE` in order to filter for specific patterns within a column. Let's continue using our DataBase to query an example of its usage.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE description LIKE "%guide%";
+----+----------------------------+----------------+----------------------------------
----------------------+--------------------+
| id | name                       | published_date | description
| category           |
+----+----------------------------+----------------+----------------------------------
----------------------+--------------------+
```

```
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture   | Defensive Security |
|  2 | Bug Bounty Bootcamp        | 2021-11-16     | The Guide to Finding and Reportin
g Web Vulnerabilities | Offensive Security |
|  3 | Car Hacker's Handbook      | 2016-02-25     | A Guide for the Penetration Teste
r                     | Offensive Security |
|  4 | Designing Secure Software  | 2021-12-21     | A Guide for Developers
| Defensive Security |
+----+----------------------------+----------------+----------------------------------
----------------------+--------------------+

4 rows in set (0.00 sec)
```

The query above returns a list of records from the books filtered, but the ones using the `WHERE` clause that contains the word guide by using the `LIKE` operator.

AND Operator

The `AND` operator uses multiple conditions within a query and returns `TRUE` if all of them are true.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE category = "Offensive Security" AND name = "Bug Bounty Bootcamp";
+----+---------------------+----------------+----------------------------------------
---------------+--------------------+
| id | name                | published_date | description
| category           |
+----+---------------------+----------------+----------------------------------------
---------------+--------------------+
|  2 | Bug Bounty Bootcamp | 2021-11-16     | The Guide to Finding and Reporting Web V
ulnerabilities | Offensive Security |
+----+---------------------+----------------+----------------------------------------
---------------+--------------------+

1 row in set (0.00 sec)
```

The query above returns the book with the name **Bug Bounty Bootcamp**, which is under the category of **Offensive Security**.

OR Operator

The `OR` operator combines multiple conditions within queries and returns `TRUE` if at least one of these conditions is true.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE name LIKE "%Android%" OR name LIKE "%iOS%";
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
| id | name                       | published_date | description
| category           |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture | Defensive Security |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
```

```
1 row in set (0.00 sec)
```

The query above returns books whose **names** include either **Android** or **IOS**.

NOT Operator

The `NOT` operator reverses the value of a boolean operator, allowing us to exclude a specific condition.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE NOT description LIKE "%guide%";
+----+-----------------+----------------+----------------------------------------+----
----------------+
| id | name            | published_date | description                            | cat
egory           |
+----+-----------------+----------------+----------------------------------------+----
----------------+
|  5 | Ethical Hacking | 2021-11-02     | A Hands-on Introduction to Breaking In | Off
ensive Security |
+----+-----------------+----------------+----------------------------------------+----
----------------+

1 row in set (0.00 sec)
```

The query above returns results where the description does not contain the word **guide**.

BETWEEN Operator

The `BETWEEN` operator allows us to test if a value exists within a defined **range**.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE id BETWEEN 2 AND 4;
+----+---------------------------+----------------+-----------------------------------
---------------------+--------------------+
| id | name                      | published_date | description
| category           |
+----+---------------------------+----------------+-----------------------------------
---------------------+--------------------+
|  2 | Bug Bounty Bootcamp       | 2021-11-16     | The Guide to Finding and Reporting
Web Vulnerabilities | Offensive Security |
|  3 | Car Hacker's Handbook     | 2016-02-25     | A Guide for the Penetration Tester
| Offensive Security |
|  4 | Designing Secure Software | 2021-12-21     | A Guide for Developers
| Defensive Security |
+----+---------------------------+----------------+-----------------------------------
---------------------+--------------------+

3 rows in set (0.00 sec)
```

The query above returns books whose **id** is **between 2** and **4**.

Comparison Operators

The comparison operators are used to compare values and check if they meet specified criteria.

Equal To Operator

The `=` (Equal) operator compares two expressions and determines if they are equal, or it can check if a value matches another one in a specific column.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE name = "Designing Secure Software";
+----+---------------------------+----------------+-----------------------+----------
----------+
| id | name                      | published_date | description           | category
|
+----+---------------------------+----------------+-----------------------+----------
----------+
|  4 | Designing Secure Software | 2021-12-21     | A Guide for Developers | Defensive
Security |
+----+---------------------------+----------------+-----------------------+----------
----------+

1 row in set (0.10 sec)
```

The query above returns the book with the **exact name Designing Secure Software**.

Not Equal To Operator

The `!=` (not equal) operator compares expressions and tests if they are not equal; it also checks if a value differs from the one within a column.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE category != "Offensive Security";
+----+---------------------------+----------------+---------------------------------
--------------------+--------------------+
| id | name                      | published_date | description
| category           |
+----+---------------------------+----------------+---------------------------------
--------------------+--------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture | Defensive Security |
|  4 | Designing Secure Software  | 2021-12-21     | A Guide for Developers
| Defensive Security |
+----+---------------------------+----------------+---------------------------------
--------------------+--------------------+

2 rows in set (0.00 sec)
```

The query above returns books **except** those whose **category** is **Offensive Security**.

Less Than Operator

Less Than Operator

The `<` (less than) operator compares if the expression with a given value is lesser than the provided one.

Terminal

```
mysql> SELECT *
    FROM books
```

```
    WHERE published_date < "2020-01-01";
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
| id | name                       | published_date | description
| category           |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture | Defensive Security |
|  3 | Car Hacker's Handbook      | 2016-02-25     | A Guide for the Penetration Teste
r                   | Offensive Security |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+

2 rows in set (0.00 sec)
```

The query above returns books that were published **before January 1, 2020**.

Greater Than Operator

The `>` (greater than) operator compares if the expression with a given value is greater than the provided one.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE published_date > "2020-01-01";
+----+----------------------------+----------------+----------------------------------
---------------------+--------------------+
| id | name                       | published_date | description
| category           |
+----+----------------------------+----------------+----------------------------------
---------------------+--------------------+
|  2 | Bug Bounty Bootcamp        | 2021-11-16     | The Guide to Finding and Reporting
Web Vulnerabilities | Offensive Security |
|  4 | Designing Secure Software | 2021-12-21     | A Guide for Developers
| Defensive Security |
|  5 | Ethical Hacking            | 2021-11-02     | A Hands-on Introduction to Breakin
g In                | Offensive Security |
+----+----------------------------+----------------+----------------------------------
---------------------+--------------------+

3 rows in set (0.00 sec)
```

The query above returns books published **after January 1, 2020**.

Less Than or Equal To and Greater Than or Equal To Operators

The `<=` (Less than or equal) operator compares if the expression with a given value is less than or equal to the provided one. On the other hand, The `>=` (Greater than or Equal) operator compares if the expression with a given value is greater than or equal to the provided one. Let's observe some examples of both below.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE published_date <= "2021-11-15";
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
| id | name                       | published_date | description
| category           |
```

```
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
|  1 | Android Security Internals | 2014-10-14     | An In-Depth Guide to Android's Se
curity Architecture | Defensive Security |
|  3 | Car Hacker's Handbook      | 2016-02-25     | A Guide for the Penetration Teste
r                   | Offensive Security |
|  5 | Ethical Hacking            | 2021-11-02     | A Hands-on Introduction to Breaki
ng In               | Offensive Security |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+

3 rows in set (0.00 sec)
```

The query above returns books **published on or before November 15, 2021**.

Terminal

```
mysql> SELECT *
    FROM books
    WHERE published_date >= "2021-11-02";
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
| id | name                       | published_date | description
| category           |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+
|  2 | Bug Bounty Bootcamp        | 2021-11-16     | The Guide to Finding and Reporting
Web Vulnerabilities | Offensive Security |
|  4 | Designing Secure Software | 2021-12-21     | A Guide for Developers
| Defensive Security |
|  5 | Ethical Hacking            | 2021-11-02     | A Hands-on Introduction to Breakin
g In                | Offensive Security |
+----+----------------------------+----------------+----------------------------------
--------------------+--------------------+

3 rows in set (0.00 sec)
```

The query above returns books that were **published on or after November 2, 2021**.

When working with Data, functions can help us streamline queries and operations and manipulate data. Let's explore some of these functions next.

String Functions

Strings functions perform operations on a string, returning a value associated with it.

CONCAT() Function

This function is used to add two or more strings together. It is useful to combine text from different columns.

Terminal

```
mysql> SELECT CONCAT(name, " is a type of ", category, " book.") AS book_info FROM boo
ks;
+-------------------------------------------------------------------+
| book_info                                                          |
+-------------------------------------------------------------------+
| Android Security Internals is a type of Defensive Security book. |
| Bug Bounty Bootcamp is a type of Offensive Security book.        |
```

```
| Car Hacker's Handbook is a type of Offensive Security book.      |
| Designing Secure Software is a type of Defensive Security book.  |
| Ethical Hacking is a type of Offensive Security book.            |
+-------------------------------------------------------------------+

5 rows in set (0.00 sec)
```

This query concatenates the **name** and **category** columns from the **books** table into a single one named **book_info**.

GROUP_CONCAT() Function

This function can help us to concatenate data from multiple rows into one field. Let's explore an example of its usage.

Terminal

```
mysql> SELECT category, GROUP_CONCAT(name SEPARATOR ", ") AS books
    FROM books
    GROUP BY category;
+--------------------+-------------------------------------------------------------------+
| category           | books                                                             |
+--------------------+-------------------------------------------------------------------+
| Defensive Security | Android Security Internals, Designing Secure Software             |
| Offensive Security | Bug Bounty Bootcamp, Car Hacker's Handbook, Ethical Hacking       |
+--------------------+-------------------------------------------------------------------+

2 rows in set (0.01 sec)
```

The query above groups the **books** by **category** and concatenates the titles of books within each category into a **single string**.

SUBSTRING() Function

This function will retrieve a substring from a string within a query, starting at a determined position. The length of this substring can also be specified.

Terminal

```
mysql> SELECT SUBSTRING(published_date, 1, 4) AS published_year FROM books;
+----------------+
| published_year |
+----------------+
| 2014           |
| 2021           |
| 2016           |
| 2021           |
| 2021           |
+----------------+

5 rows in set (0.00 sec)
```

In the query above, we can observe how it extracts the first **four** characters from the **published_date** column and stores them in the **published_year** column.

LENGTH() Function

This function returns the number of characters in a string. This includes spaces and punctuation. We can find an example below.

Terminal

```
mysql> SELECT LENGTH(name) AS name_length FROM books;
+-------------+
| name_length |
+-------------+
|          26 |
|          19 |
|          21 |
|          25 |
|          15 |
+-------------+

5 rows in set (0.00 sec)
```

As we can observe above, the query calculates the length of the string within the **name** column and stores it in a column named **name_length**.

Aggregate Functions

These functions aggregate the value of multiple rows within one specified criteria in the query; It can combine multiple values into one result.

COUNT() Function

This function returns the number of records within an expression, as the example below shows.

Terminal

```
mysql> SELECT COUNT(*) AS total_books FROM books;
+-------------+
| total_books |
+-------------+
|           5 |
+-------------+

1 row in set (0.01 sec)
```

This query above counts the total number of rows in the **books** table. The result is **5**, as there are five books in the books table, and it's stored in the **total_books** column.

SUM() Function

This function sums all values (not NULL) of a determined column.

Note: There is no need to execute this query. This is just for example purposes.

Terminal

```
mysql> SELECT SUM(price) AS total_price FROM books;
+-------------+
| total_price |
+-------------+
|      249.95 |
+-------------+
```

```
1 row in set (0.00 sec)
```

The query above calculates the total sum of the **price** column. The result provides the aggregate price of all books in the column **total_price**.

MAX() Function

This function calculates the maximum value within a provided column in an expression.

Terminal

```
mysql> SELECT MAX(published_date) AS latest_book FROM books;
+-------------+
| latest_book |
+-------------+
| 2021-12-21  |
+-------------+

1 row in set (0.00 sec)
```

The query above retrieves the latest publication (maximum value) date from the **books** table. The result **2021-12-21** is stored in the column **latest_book**.

MIN() Function

This function calculates the minimum value within a provided column in an expression.

Terminal

```
mysql> SELECT MIN(published_date) AS earliest_book FROM books;
+--------------+
| earliest_book |
+--------------+
| 2014-10-14    |
+--------------+

1 row in set (0.00 sec)
```

The query above retrieves the earliest publication (minimum value) date from the **books** table. The result **2014-10-14** is stored in the **earliest_book** column.